Universität Zürich
Institut für Informatik

HS 2025

Student Name:
Matrikel-Nr:

Numerical Methods in Informatics - Exercise 4

Assigned: 11.11.2025 - Due: 24.11.2025 (end of day)

Please upload your solutions to the OLAT system.

Practice

In many real-world scenarios, you often get some noisy data, which doesn't perfectly represent the phenomenon you're observing. For instance, consider the data points illustrated in Figure 1, derived from a measured linear relationship.

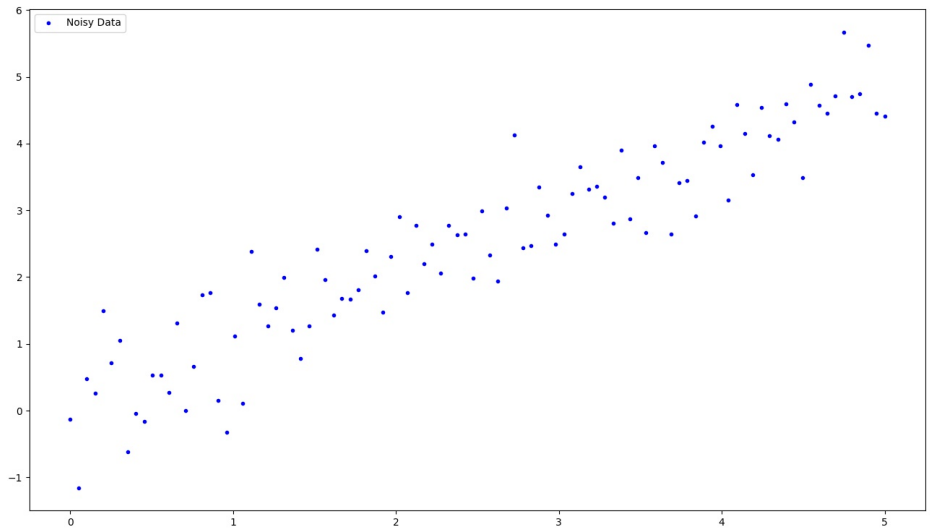

Figure 1: Noisy data.

Fitting a polynomial through this data, will result in a highly oscillating curve, as you've

already seen in an earlier lecture. Interpolating with some sort of spline will reduce the oscillation, but still result in a curve passing through all those points, which is undesired. Here, the "general direction" is interesting, as shown in Figure 2.

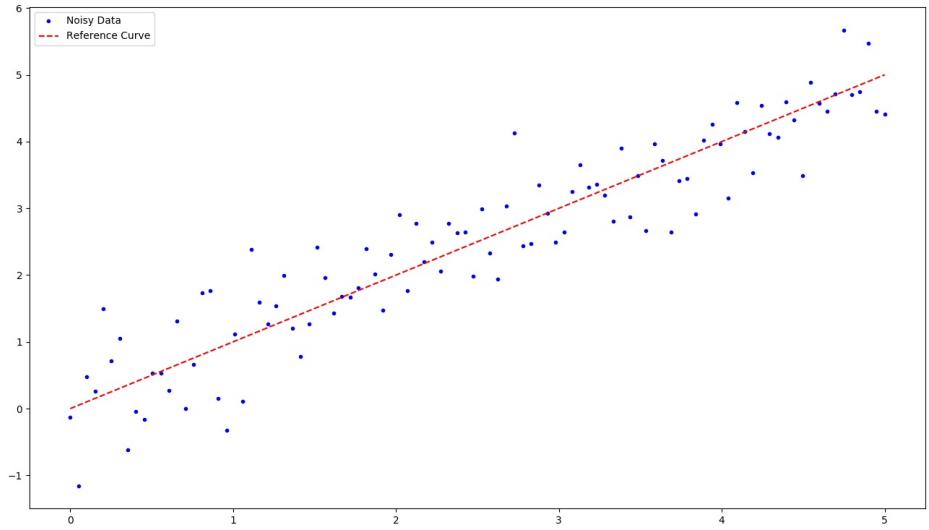

Figure 2: Noisy data (blue) with a reference curve (red, dashed).

This "general direction" can also be mathematically formulated as the eigenvector with the largest eigenvalue extracted from the covariance matrix of the given data.

Shifting Data:

Vectors are originated in $(0, 0)$. Thus, to calculate the direction within the given data, the data has to be shifted by its mean. This centers the data around the origin, enabling a vector calculated from the data to represent an internal orientation within the dataset.

Eigendecomposition:

Calculating the eigenvector with the largest eigenvalue will now result in the "principal direction" of the (mean-shifted) data. The result is shown in Figure 3.

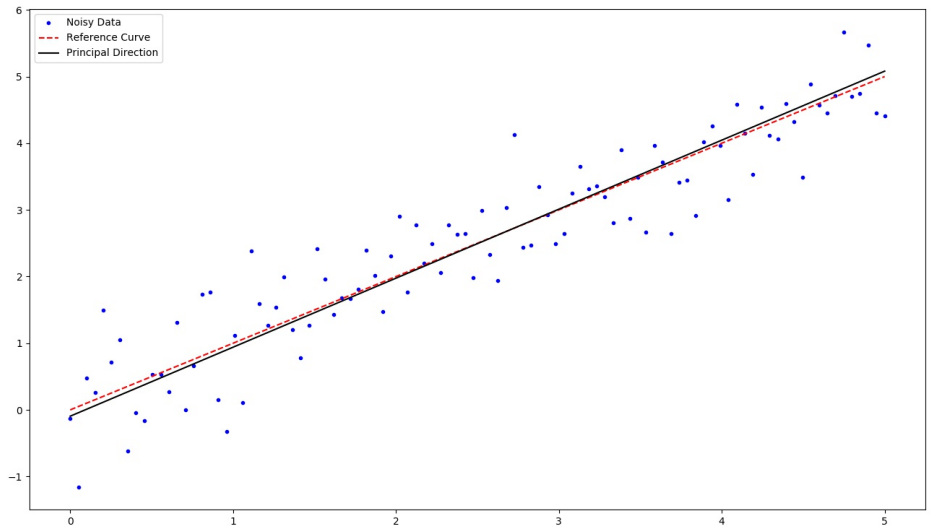

Figure 3: Noisy data (blue) with a reference curve (red, dashed) and the estimated principal direction (black).

4.1 Eigendecomposition

a) (30 Min, 2 Points) Power Method

Please implement the function `powerMethod(A, b)` in `backend.py`. The method is supposed to calculate the largest eigenvector (with respect to absolute eigenvalues) of A using b as a starting point.

b) (30 Min, 2 Points) Inverse Power Method

Please implement the function `inversePowerMethod(A, b)` in `backend.py`. The method is supposed to calculate the smallest eigenvector (with respect to absolute eigenvalues) of A using b as a starting point.

c) (30 Min, 6 Points) Linear Approximation via Eigenvectors

Please implement the function `linearPCA(x, y)` in `backend.py`, which performs the above described line fitting algorithm on the given data.

Hints:

- If you don't want to use your own implementation of the power method, you can use numpy's eigendecomposition. This will always return all eigenvalues and eigenvectors, which may take longer than just calculating the largest one.
- D, as constructed from above is positive-(semi)-definite, which means, you can use the `np.linalg.eigh` method, instead of `np.linalg.eig`.

Hand-in requirements:

Please submit only your backend.py. Zipped files or other file types will not be assessed.